

1

AI 기반 KCMVP 사전 적합성 검증 도구

2026.02.25 부채널 공모전 Project L3

로드맵 및 개요 작성

2394025 조수빈

AI 기반 KCMVP 사전 적합성 점검 도구

Target
Open Source Crypto Modules

! 배경 및 문제의식

- 📄 문서 vs 구현 불일치:
보안정책서와 실제 소스코드 간 괴리 발생
- 🚫 금지/권고 미준수:
위험한 난수 생성, 디버그 혹 등 필수 요건 누락
- 🔄 반복적 반려:
검증기관과의 커뮤니케이션 비용 과다 발생

🎯 연구 목표

사전 점검(Pre-check) 자동화

KCMVP 문서/가이드라인을 기반으로
오류 탐지 및 수정 제안(Patch) 을 자동 생성하여
반려 요인을 사전에 제거

🔍 핵심 과제 & 연구 질문(RQ)



하이브리드 탐지 (RQ1)

정규식/AST 를 기반 탐지 + LLM 문맥 해석
→ 부적합 요소를 유의미하게 탐지하는가?



수정 마크다운 생성 (RQ2)

근거, 위치, 권고 조치가 포함된 리포트 생성
→ 개발자가 "즉시 조치 가능"한 수준인가?



효율성 검증 (RQ3)

프로토타입 적용 및 정량/정성 평가
→ 질의/보완 루프 횟수를 줄일 잠재력 평가

📁 최종 산출물

- 📜 KCMVP Pre-check Rule (v1)
- 🔧 Prototype Tool (CLI)
- 📄 REPORT & Patch Suggestion

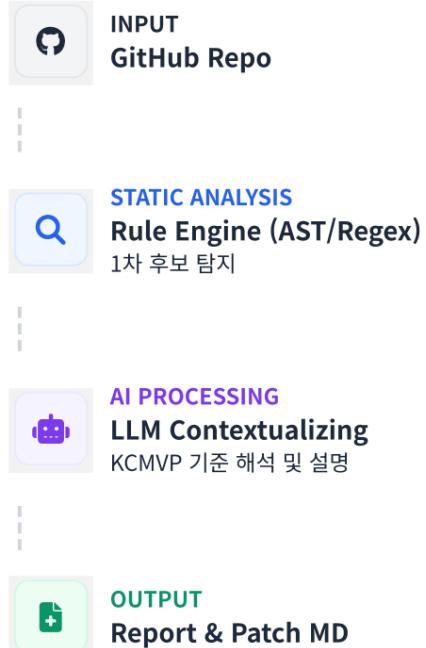
💡 기대 효과

단순 규정 검색이 아닌,
실무적 코드 수정안을 제시하여
개발자 편의성 극대화

방법론 및 4주 실행 계획

구현 파이프라인 설계 및 주차별 마일스톤

분석 파이프라인



검증 및 평가

Quantitative
룰 탐지 정확도
Precision / Recall 측정

Qualitative
조치 용이성
개발자 관점 3점 척도

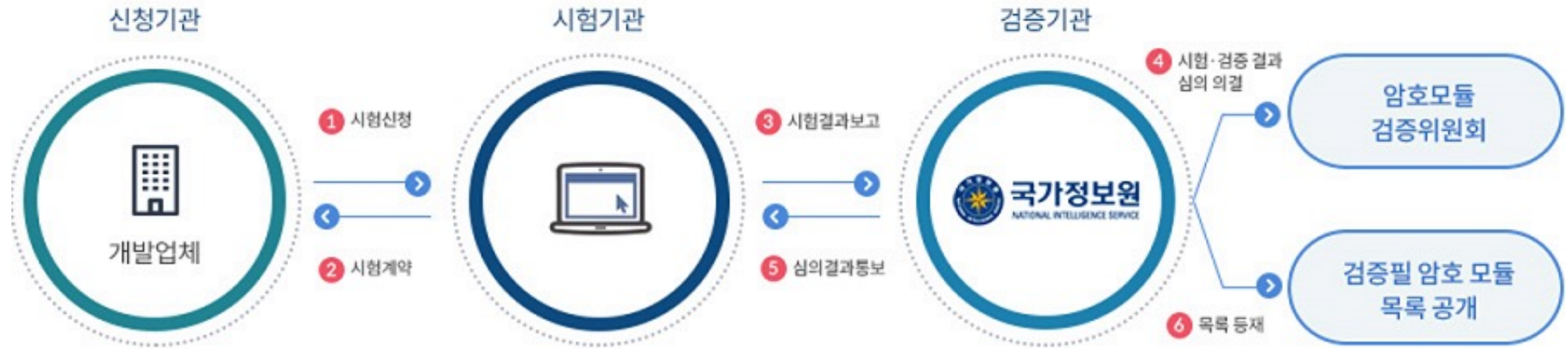
4주 실행 계획 (Weekly Plan)



1. KCMVP

- **한국형 암호모듈 검증제도(KCMVP)**
 - (KCMVP, Korea Cryptographic Module Validation Program)
 - 2009년부터 시행하고 있으며, 정보보호를 위해 국가·공공기관에서 비밀이 아닌 업무 자료를 보호하기 위해서는 KCMVP에서 인증받은 암호모듈의 사용을 요구하고 있음
- **검증필 암호모듈**
 - 암호 기술이 안전하게 구현되었는지 국가(국정원)가 확인하고 승인한 제품
 - KCMVP 인증 통과·표준 암호 알고리즘 사용이 통과되어야 함
 - 국가·공공기관 사업에서는 검증필 암호모듈 필수 탑재가 원칙

2. 기존 KCMVP 검증 절차의 문제



- 업체 내부에서 개발하고 문서를 준비하는 시간에 상당한 시간 소요
- 기관에서 검증 받는 기간이 평균 6개월 ~ 1년의 상당한 시간 소요
- 만약 보완 요구나 반려가 된다면 또다시 이를 수정하여 재시험 및 검증을 받아야 함
- 이를 다른 기업에 맡겨서 보완하려고 하면 매우 큰 비용이 소모됨
- 따라서 사전 점검을 통해 우리의 프로젝트 도구가 제출 전 단 1~2회의 보완 요구만 줄여주더라도, 출시 기간과 인건비를 매우 단축할 수 있음

3. 구현 목표

- AI 기반 KCMVP 사전 적합성 점검 도구
 - KCMVP 가이드라인을 학습한 AI → 소스코드를 분석 → 보안 결함 찾아서 보고서와 수정패치(.md) 제안
- 제출물
 - AI가 참조할 KCMVP 기반 룰셋
 - 프로토타입 툴 (아마 웹)
 - 툴을 통한 수정패치 예시 (실제 툴을 통한 출력물)
 - 최종 보고서 (성능 평가 포함) → 논문으로 고도화 진행해야 함
- 대상
 - 대칭키 암호 우선 진행 + LEA로 진행

4.1. 구체화: AI 룰셋 설계

- 여러 개 암호 한다고 가정

1) 공통 보안 룰셋

- 알고리즘이나 운영 모드에 상관 없이 KCMVP 인증을 위해 반드시 지켜야할 룰
- Ex) 잔존 정보 제거 여부, 에러 처리 여부

2) 알고리즘 공통 룰셋

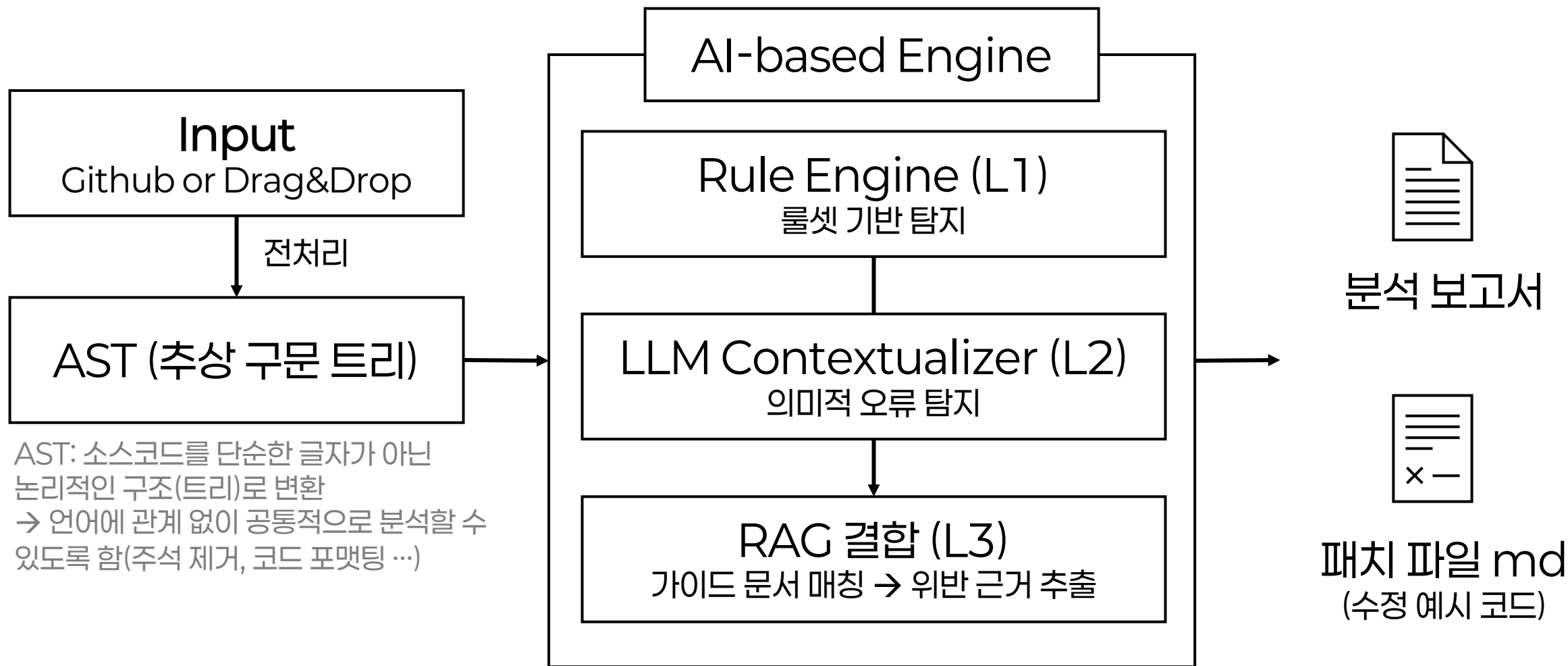
- 같은 알고리즘 내에서 알고리즘 자체의 구현이 잘 지켜졌는지 여부에 대한 룰
- Ex) 키 스케줄링 올바른가? 라운드 연산이 올바른가? 여부

3) 운영 모드 특화 룰셋

- 공통 알고리즘(ex.LEA) 내에서도 다양한 운영 모드가 존재하므로, 운영 모드에 따른 운영 모드 특화 룰
- Ex) CBC 모드에서는 IV의 랜덤성, CTR에서는 Nonce의 유일성...

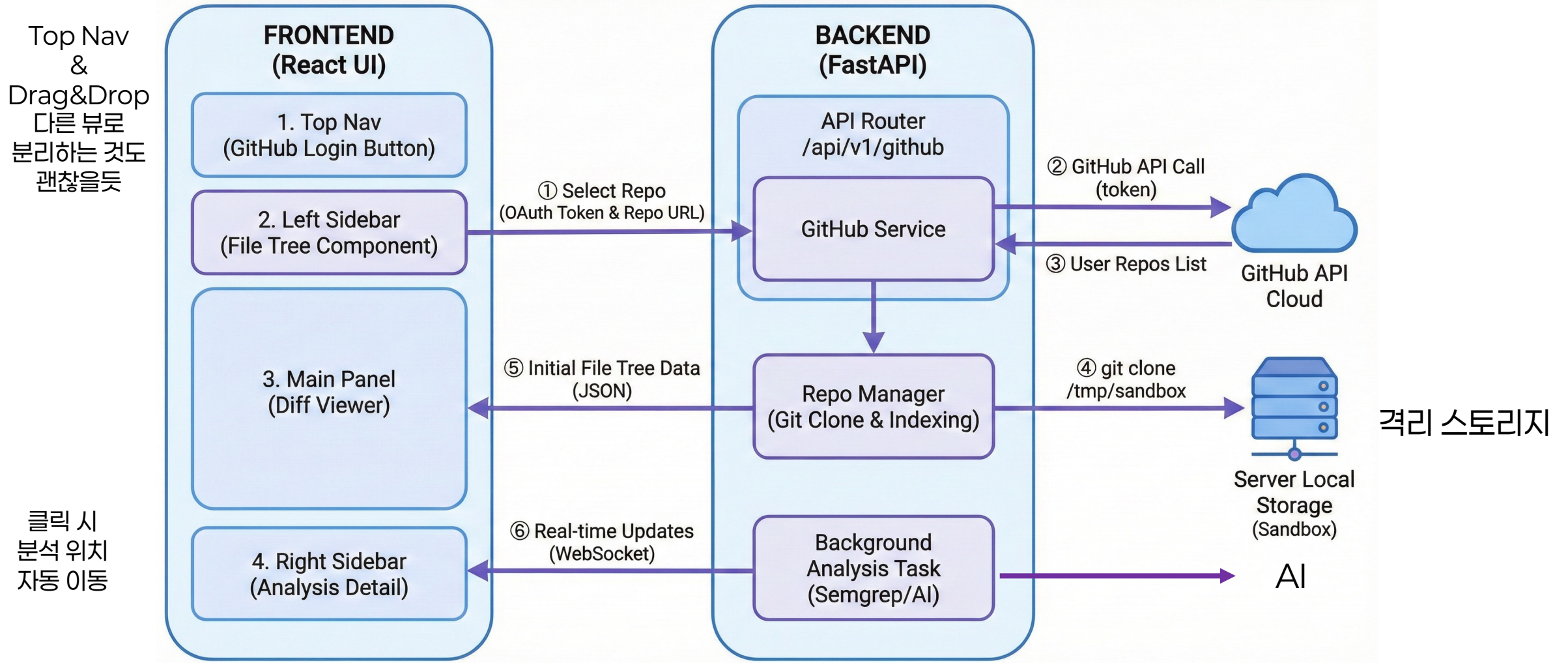
4.2. 구체화: 파이프라인

RAG: 검색 증강 생성 → 필요한 정보가 DB에 저장되어 있으면,
AI가 그 DB에서 원하는 문구를 찾아옴
→ 매번 모델 학습할 필요 없이 DB에 가이드라인 업데이트하면
된다는 이점 존재



4.3. 구체화: 프론트-백엔드 연동 구조

IDE 구성이랑 비슷하게 프론트 짜도 괜찮을 것 같음



5.1. 계획: 일정

일정

일단 이번주? 까지 교수님께 진행 사항 보고를 해야 해서

- [1] ~2월 27일(금): 지금 기획안 바탕으로 진행해서 가능성 확인
- [2] ~3월 초: LEA + ARIA 정도 완성 + 웹 연동
- [3] ~ 3월 중순: 데이터셋 구상 및 실험 완료 + 오류 해결
- [4] 3월 말 ~ 4월 초: 보고서 작성 → 논문으로

- 시간 되면: AES + a
- 시간 안되면: LEA만 일단
- 그리고 시간이 너무 촉박할 것 같아서 학부연구생도 아예 주제를 지금처럼 하나씩 맡아서 진행하고 서로 피드백 받는 형식으로 진행하는 게 나을 것 같습니다

5.2. 계획: 역할 분담

지금 문제가

(1) 대칭키 하나 룰셋 설정하고 돌려보는데 얼마나 걸리느냐

- 이거는 제가 아직 해보지 않아서 금요일까지 가능성을 봐야 알 것 같습니다.

[후보 1: (1) 이 쉬울 경우]

- 그러면 최대한 빨리 대칭키 하나 해보고 파이프라인 완료 → 한 명씩 대칭키 하나씩 맡아서 룰셋 세팅하고 실험 각자 돌려서 데이터 많이 쌓기
- 이 경우 역할 분담: 한 명당 대칭키 하나씩, 문서 작성 / 프론트 이렇게 나눔

[후보 2: (1) 이 어려울 경우]

- 그냥 대칭키 하나(LEA)로만 진행 → 난이도 확인 후 추후 역할 분담...

6. 참고 사이트 정리

- 1) 국가사이버안보센터-암호모듈검증:
<https://www.ncsc.go.kr/PageLink.do>
- 2) KISA 암호이용활성화-암호모듈검증:
<https://seed.kisa.or.kr/kisa/kcmvp/EgovSummary.do>
- 3) KISA 암호이용활성화-국산암호기술-LEA(논문 및 규격서 포함):
<https://seed.kisa.or.kr/kisa/algorithm/EgovLeaInfo.do>
- 4) 국가보안기술연구소. (2015). 블록암호 LEA 소스코드 사용 매뉴얼 (v1.0) : LEA 이걸로 보면 좋을듯 해요(교수님이 주신 자료에 있음)
- 5) 미래창조과학부, 한국인터넷진흥원. (2013). 암호기술 구현 안내서 (KISA 안내해설 제2011-9호)

2

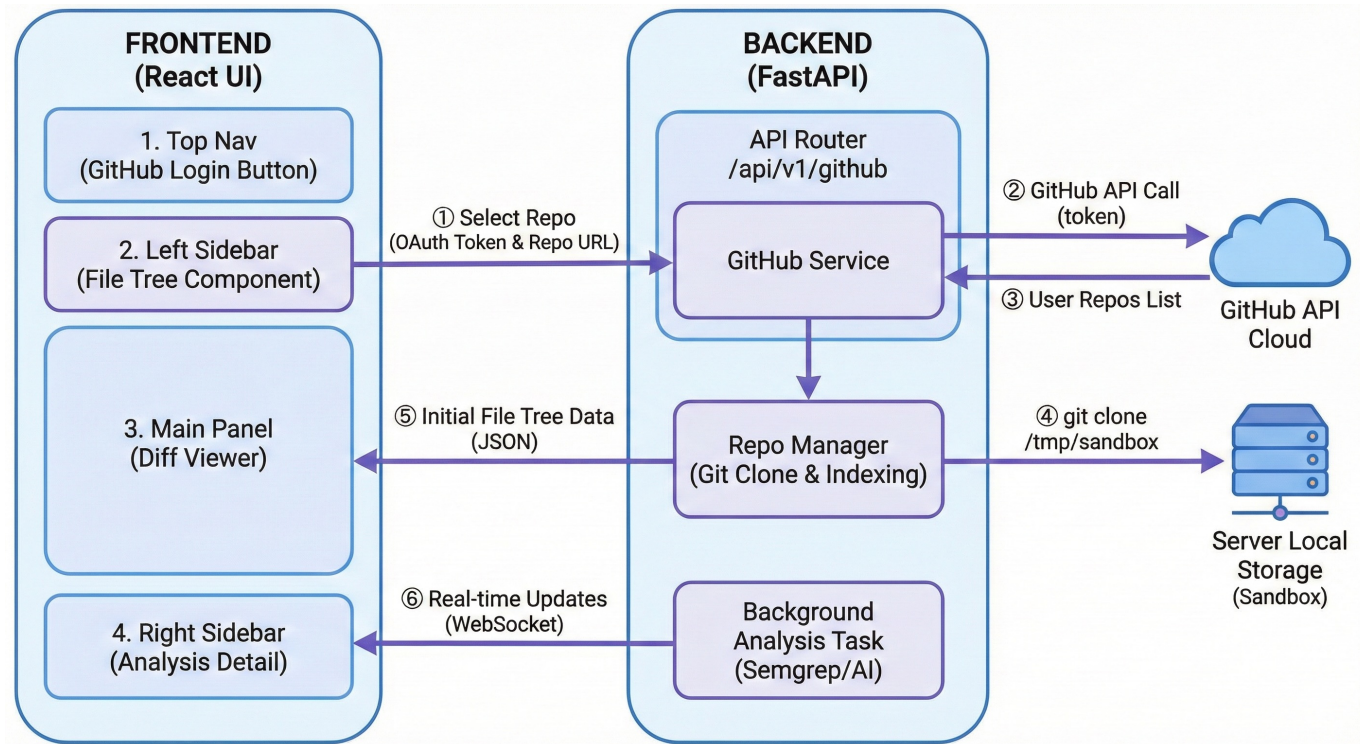
AI 기반 KCMVP 사전 적합성 검증 도구

2026.02.25 부채널 공모전 Project L3_2

진행 사항 공유

20260306 금요일 회의

진행 사항 요약: LEA로 진행하기로 함



- UI 설계는 대략적으로 완료

→ 추후 시간 남으면 AI 티 만나게 +
디테일한 부분 완성할 예정

- 깃허브 연동 x 일단은
zip으로 올리도록 설정

→ 테스트 완료 후 연동할 예정

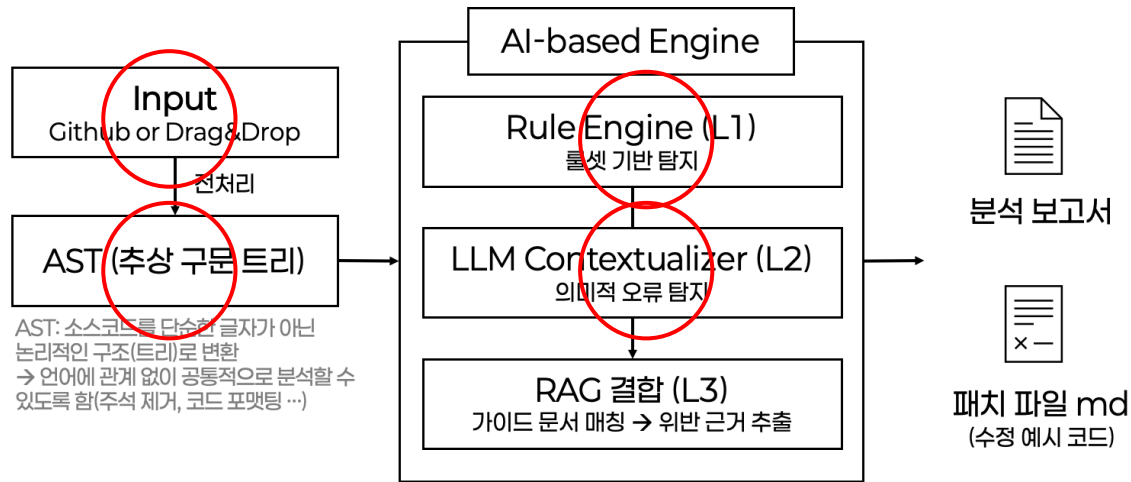
- 디테일 구현

→ 문서 - 코드 분리

→ 오류 위치 확인

→ IDE 유사 디자인

진행 사항 요약: LEA로 진행하기로 함



부채널 공모전: AI 기반 KCMVP 사전 점검

8

- 동그라미 친 부분: 백엔드 설계까지는 완료

→ L1은 구체화 진행 중

→ 깃허브 연동은 아직 진행 X

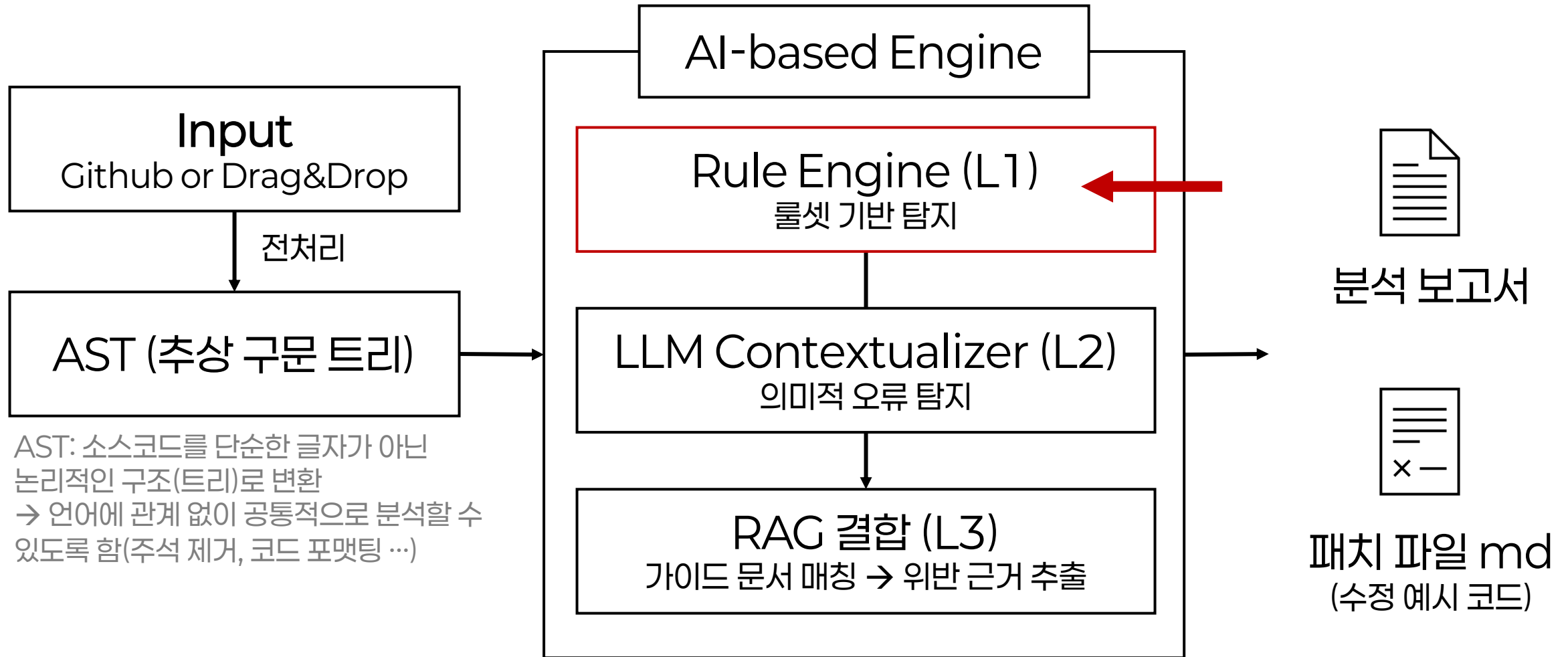
AI는 gemini-flash-lite-latest 사용(무료), 하루 20번 제한 존재

→ 그래서 서류 전체를 검토하는 건 어려울 것이라 판단 → L1에서 폭 넓게 체크

→ L2가 맥락을 보고 체크한 부분이 실제 문제인지, 맥락상 통과되는지 확인

→ L3에서 최종 확인하고 근거 인용

L1 구체화



L1 구체화 코드 부분 (초안)

1) 공통 보안 룰셋

- 알고리즘이나 운영 모드에 상관 없이 KCMVP 인증을 위해 반드시 지켜야할 룰
- Ex) 잔존 정보 제거 여부, 에러 처리 여부

2) 알고리즘 공통 룰셋

- 같은 알고리즘 내에서 알고리즘 자체의 구현이 잘 지켜졌는지 여부에 대한 룰
- Ex) 키 스케줄링 올바른가? 라운드 연산이 올바른가? 여부

3) 운영 모드 특화 룰셋

- 공통 알고리즘(ex.LEA) 내에서도 다양한 운영 모드가 존재하므로, 운영 모드에 따른 운영 모드 특화 룰
- Ex) CBC 모드에서는 IV의 랜덤성, CTR에서는 Nonce의 유일성...

L1 룰셋 구체화 1

backend/rules/

- common/
 - security.yaml # 코드: 알고리즘 공통 보안 룰
- algorithm/
 - lea.yaml # 코드: LEA 알고리즘 룰
 - aria.yaml # 코드: ARIA 알고리즘 룰 → 예시를 위한 것이고 사용 안할 것
- mode
 - cbc.yaml # 코드: CBC 모드 룰
 - ctr.yaml # 코드: CTR 모드 룰 + 추가 예정
- docs/
 - design.yaml # 문서: 기본/상세 설계서용 문서 룰(DOC-0xx)
 - config_mgmt.yaml # 문서: 형상관리 문서용 문서 룰(DOC-2xx)
 - test.yaml # 문서: 시험서(작성 안내서)용 문서 룰(DOC-1xx)
- traceability/
 - traceability.yaml # 설계-코드-시험 추적성 룰(TRC-xxx)

사용할 문서 정리

문제: 암호모듈 요구 사항이 유료

KSSN(한국표준정보망) KS X ISO /IEC 19790 – 암호모듈 보안 요구사항:

➔ 암호 모듈을 위한 보안 요구사항 안내

• KSSN(한국표준정보망) KS X ISO/IEC 24759 – 암호모듈 시험 요구사항:

➔ 명세된 요구사항에 대해 시험하기 위해 시험기관이 사용하는 시험방법을 명세

아래 문서들로 대체 예정

• KISA 암호이용 활성화: 암호모듈 제출물 작성 안내서(2025. 9.):

<https://seed.kisa.or.kr/kisa/Board/203/detailView.do>

• ➔ 암호모듈 제출물 작성을 위한 구현 안내서

• LEA 소스코드 사용 매뉴얼(v 1.0, 국보연) / 128비트 블록암호 LEA 규격서

화면

KCMVP Pre-Validator

암호 모듈 정적 분석 도구

GitHub 링크나 ZIP 파일을 업로드하면 KCMVP 룰로 소스를 분석하고, 위반 위치와 설명을 IDE 형태로 보여줍니다.

ZIP 파일(.zip)만 업로드할 수 있습니다.

GitHub 레포 분석

공개 레포지토리 URL을 붙여넣고 분석을 시작합니다. (현재는 ZIP 우선 지원)

GitHub 링크로 분석

아래 한 줄에서 코드 ZIP과 3종 문서 PDF(설계서 / 형상관리 / 시험서)를 함께 업로드하세요.

ZIP(코드) 1개 + PDF(문서) 3개까지 업로드할 수 있습니다. 문서는 텍스트 기반 PDF를 권장합니다.

ZIP 파일로 분석

화면

KCMVP 사전 점검

새 분석

GitHub URL 또는 owner/repo

파일 선택

선택된 파일 없음

알고리즘

모드

ZIP: ast_test.zip

파일 7개

위반 4건 (H 4 · M 0 · L 0)

분석 완료

설계서

형상관리 문서

시험서

코드

종합

파일 트리

src/hardcoded_key.c

src/uses_secure_clear.c

src/has_explicit_bzero.c

src/no_clear.c

src/secure_clear.c

src/has_memset.c

include/secure_clear.h

선택된 파일: src/hardcoded_key.c

전체 파일 AST 보기

파일 전체 흐름 보기

이 파일에는 파일 전체 수준의 위반 (COM-001 등)이 있습니다. 잔존 정보 제거 코드가 전혀 없거나, 가이드라인에서 요구하는 처리 로직이 누락되어 있을 수 있습니다.

```
1
2 static const unsigned char KEY[16] = {
3     0x01, 0x23, 0x45, 0x67, 0x89, 0xAB, 0xCD, 0xEF,
4     0xFE, 0xDC, 0xBA, 0x98, 0x76, 0x54, 0x32, 0x10
5 };
6 void use_hardcoded(void) {
7     encrypt_block(KEY);
8 }
9
```

요약

위반 목록

패치

분석 완료

COM-001

src/hardcoded_key.c:전체

잔존 정보 제거

COM-001

src/no_clear.c:전체

잔존 정보 제거

COM-001

include/secure_clear.h:전체

잔존 정보 제거

COM-003

src/hardcoded_key.c:3

하드코딩된 키/IV 금지

진행 목표 및 역할 분담

- ~3월 16일 (학부생 회의):
 - 룰셋 설계 어느정도 완료
 - RAG 파이프라인 연결 및 AI 최적화 완료
 - 출력될 최종 보고서 형태 형식 구성 완료
- 현재 역할 분담
 - 룰셋 설계를 LEA 알고리즘 파트 / 서류 관련 파트로 나눠서 진행

Q

- AI 부분: 로컬에서 해도 괜찮
- 서류부분: 소프트웨어 기본적 (security 1 이면서 소프트웨어인 것만 (펌웨어 x)) → 이 부분도 크롭해서 넣도록 하라
- 진행도 : 4페이지 이내(양식)
- 논문: phase그림 다이어그램 ★ + 화면 포함 (선택), 참고문헌
할루시네이션을 조심하라!!! (이중 검증을 하라...)

3

AI 기반 KCMVP 사전 적합성 검증 도구

2026.02.25 부채널 공모전 Project L3_2

진행 사항 공유

20260310 월요일 회의

피드백 진행사항 1

- 교수님의 피드백
 - AI 부분: 로컬에서 해도 괜찮
그래서 파인튜닝을 해야하는데 아직 못했습니다
 - 서류부분: 소프트웨어 기본적 (security 1 이면서 소프트웨어인 것만 (펌웨어 x))
이미 문서 자체가 보안수준 1에 해당하는 소프트웨어 암호모듈을 대상으로 함

1 | 개요

본 가이드 문서는 KS X ISO/IEC 19790:2015, KS X ISO/IEC 24759:2015 표준문서에 근거하여 국내 암호모듈 검증제도에서 요구되는 기본 및 상세설계서를 작성하는데 참고할 수 있는 내용을 담고 있다. 해당 표준에서 정의한 보안수준 1에 해당하는 소프트웨어 암호모듈로서 갖추어야 할 보안요구사항들에 대해 기본 및 상세설계서를 작성하는 방법을 안내한다.

피드백 진행사항 1

→ 이 부분도 크롭해서 넣도록 하라??

- 진행도 : 4페이지 이내(양식)

아마 역할 분담하면 이틀 만에도 진행할 수 있음 (프레임워크 완성 → 실험 → 논문)

- 논문: phase그림 다이어그램 ★ + 화면 포함 (선택), 참고문헌 할루시네이션을 조심하라!!! (이중 검증을 하라...)

→ 다이어그램은 드롭박스 ppt 내에 만들예정

LEA 룰셋 가이드라인 예시 1

- 자세한 내용은 드롭박스 KCMVP/룰설계 가이드라인 자료 마크다운 파일에 있음 (가독성이 별로면, 마크다운 뷰어로 보는 것을 추천)
- 예시 이미지는 LEA 표준 규격서 + 소스코드 사용법에서 가져옴

5.1. 규격

본 규격서에서는 LEA를 각 키 길이에 따라 LEA-128, LEA-192, LEA-256으로 구분한다. 블록암호 LEA의 규격은 <표 5-1>과 같이 정리할 수 있다. 블록 길이를 Nb 바이트, 비밀키 길이를 Nk 바이트, 라운드 수를 Nr로 나타내었다.

<표 5-1> LEA 규격

구분	Nb	Nk	Nr
LEA-128	16	16	24
LEA-192	16	24	28
LEA-256	16	32	32

→ LEA-128의 경우, Nr(라운드 수)가 24여야 함. 즉

```
rules:
- id: LEA-001
  category: algorithm
  algorithm: LEA
  name: "LEA-128 라운드 수 준수"
  description: "LEA-128 구현에서 라운드 수(Nr)가 표준 규격인 24회로
정의되어 있는지 검사함"
  pattern_type: "regex"
  pattern: "#define\\s+LEA128_NR\\s+24"
  severity: high
  kcmvp_ref: "LEA 표준 §5.1"
```

LEA 룰셋 가이드라인 예시 2

- 키 스케줄 함수에서 아래와 같은 상수를 정상적으로 사용해야 하므로

5.2.2. 암호화 키 스케줄

키 K 로부터 암호화 과정에 필요한 Nr 개의 192비트 암호화 라운드키 RK_i^{enc} ($0 \leq i \leq (Nr - 1)$)들을 생성하는 키 스케줄 과정을 설명한다.

키 스케줄 함수에서 사용되는 32비트 상수들 $\delta[i]$ ($0 \leq i \leq 7$)는 다음과 같다.

$\delta[0] = c3efe9db,$	$\delta[1] = 44626b02,$
$\delta[2] = 79e27c8a,$	$\delta[3] = 78df30ec,$
$\delta[4] = 715ea49e,$	$\delta[5] = c785da0a,$
$\delta[6] = e04ef22a,$	$\delta[7] = e5c40957.$

128비트 키 $K = (K[0], K[1], \dots, K[15])$ 에 대해 LEA-128의 암호화를 위해 사용되는 키 스케줄 함수 $KeySchedule_{128}^{enc}$ 는 24개의 192비트 암호화 라운드키

$$RK_i^{enc} = (RK_i^{enc}[0], RK_i^{enc}[1], \dots, RK_i^{enc}[5]) \quad (0 \leq i \leq 23)$$

를 알고리즘 3과 같이 생성하며, 이 과정에서 128비트 내부상태 변수 $T = (T[0], T[1], T[2], T[3])$ 가 사용된다.

rules:

```
- id: LEA-002
category: algorithm
algorithm: LEA
name: "표준 상수 delta[0] 사용 확인"
description: "키 스케줄 함수 내부에서 표준 상수
0xc3efe9db가 정상적으로 사용되는지 검사"
pattern_type: "missing"
pattern: "0xc3efe9db"
severity: medium
kcmvp_ref: "LEA 표준 §5.2.2"
```

+ 추가적으로 pattern에는 and 연산자같은 게 없어서, 저 모든 상수들을 확인해서 확실히 하고 싶다면 룰을 8개 만들어서 하는 방법으로 진행할 수 있음 (각각의 룰에 패턴을 적는 방식으로...)

LEA 룰셋 가이드라인 예시 3

- 숫자로만 특정하기 어려운 숫자이므로, ast구조를 통해서 원하는 함수나 벡터 안에 들어가고 있는지 판단해야 함.

매개 변수

ct [out]
암호문

pt [in]
암호화하려는 평문

pt_len [in]
평문의 길이(바이트). 16의 배수의 길이만 입력 가능하다.

iv [in]
CBC 모드에서 사용할 IV. 16바이트를 입력해야 한다.

key [in]
lea_set_key 함수를 통해 설정된 LEA_KEY 구조체의 주소

설명

입력된 평문을 CBC 모드를 이용해 암호화한다. 평문의 바이트 길이는 16의 배수이어야 한다.

```
rules:  
- id: CBC-C-002  
  category: mode  
  mode: CBC  
  name: "IV 배열 크기 고정 검사"  
  description: "CBC 모드에서 사용되는 IV는 반드시 16바이트 크기의 배열로 선언되어야 함"  
  pattern_type: "ast"  
  severity: high  
  kcmvp_ref: "매뉴얼 §4.3.4"
```

설계서 룰셋 가이드라인 예시 1

- 반드시 인가되지 않은 CSP에 대한 보호방법 명세가 필요하다고 하였으므로, CSP키워드 포함 + 주위 맥락 혹은 내용을 살펴서 이 부분이 보호방법 명세인지 확인이 필요함

항목	AS09.01
보안요구사항 개요	암호모듈 내의 CSP에 대한 보호
VE09.01.01	암호모듈 내에서 인가되지 않은 접근, 사용, 노출, 변경 및 대체로부터 모든 CSP에 대한 보호방법 명세 필요
작성예시	
다음은 KISACrypto V1.0 암호모듈 내에서 섹션될 수 있는 핵심 보안매개변수이다. KISACrypto V1.0 암호모듈은	

```
- id: DOC-011
doc_type: design
name: "CSP에 대한 보호방법 명세 필요"
description: " 암호 모듈 내에서 인가되지 않은 접근, ...로부터 모든 CSP에 대한
보호 방법 명세가 필요"
pattern_type: "semantic"
pattern: "CSP | ~..."
severity: medium
kcmvp_ref: "2025 최신 암호모듈 제출 안내서 AS09.01의 암호모듈 내의
CSP에 대한 보호"
```

설계서 룰셋 가이드라인 예시2

항목	AS02.22
보안요구사항 개요	동작모드간, 서비스 간 CSP에 대한 독립적 분리 필요
VE02.22.01	동작모드/서비스 별 사용되는 CSP 명세 필요
VE02.22.02	동작모드/서비스 간 CSP가 분리되는 방법 명세 필요
작성예시	
KISACrypto V1.0은 검증대상 동작모드만을 지원하며, 검증대상 동작모드에서 지원하는 검증대상 암호알고리즘 및 관련 CSP 목록은 AS02.20에서 제시하였다.	
해설	
만약, 하나의 암호모듈에서 검증대상 동작모드와 비검증대상 동작모드를 지원하는 경우 VE02.22.01에서 검증대상 동작모드와 비검증대상 동작모드에서 사용되는 암호알고리즘에 대한 CSP 목록을 제시해야하며, 또한, 검증대상 동작모드에서 비검증대상 동작모드로 천이할 경우, 사용되던 검증대상 동작모드의 CSP를 분리 사용되는 방법을 제시해야 한다. 암호모듈은 기본적으로 동작 전 자가시험을 거쳐, 성공 시, 검증대상 동작모드로 자동천이되어야 하며, 관리자가 동작모드 변경 함수를 통해 비검증대상 동작모드로 천이될 수 있다. 모드간 천이 시에는 기존의 모드에서 사용하던 CSP에 대해서는 천이될 동작모드와 독립적으로 분리할 수 있는 방법을 제시해야하며, 대표적인 방법으로는 제로화가 사용될 수 있다.	

```
- id: DOC-030
  doc_type: design
  name: "동작모드·서비스 간 CSP 독립적 분리 요구사항"
  description: "설계서에 동작모드 간·서비스 간 CSP에 대한 독립적 분리가 필요하다는 보안요구사항 개요가 포함되어야 함 (AS02.22)"
  pattern_type: "missing"
  pattern: "동작모드.*CSP|서비스.*CSP|CSP.*독립|CSP.*분리"
  severity: high
  kcmvp_ref: "2025 최신 암호모듈 제출물 작성 안내서 AS02.22 - 보안요구사항 개요"
```

→ 단어가 확정이 될 수 있을 것 같은 단어라 missing으로 하긴 했는데, 확실하게 하려면 semantic이나 ast도 괜찮음

설계서 가이드라인 도움이 될만한...

- GOOGLE – NOTEBOOK LM
 - 설계서나 알고리즘 규격서 같은 경우에는 길어서 다 못 읽고
 - 룰셋도 설계해달라고 하니까 좀 이상하게 설계해주는 경우가 많았음
 - NOTEBOOK LM은 인식 범위도 좀 넓은 것 같고 + 설계서 관련 내용해서 공부하기도 좋은 것 같아서 추천합니다



Notebook LM 1

2025 Cryptographic Module Submission Documentation Guide

+ 노트북 만들기

분석

공유

설정

PRO

수빈

출처

+ 소스 추가

웹에서 새 소스를 검색하세요

웹

Fast Research

모든 소스 선택

2025 최신_암호모듈_제출물_작성_안내서...

DOC_설계서_를_작성_가이드.md

로켓설계예시.pdf

채팅

무결성 기법: HMAC-SHA256, RSA-PSS에 관해 이 소스에서 말하는 내용을 소프트웨어/펌웨어 보안(이)라는 더 큰 맥락에서 논의하세요.

암호모듈의 '소프트웨어/펌웨어 보안' 영역이 가지는 핵심 목적은 모듈을 구성하는 소프트웨어 및 펌웨어 요소들이 인가되지 않은 변경이나 훼손으로부터 보호됨을 보증하는 것입니다 1. 이를 달성하기 위해 암호모듈은 반드시 전자서명이나 MAC(메시지 인증 코드)과 같은 **검증대상 무결성 검증방법**을 적용해야 하며, 모듈이 서비스를 시작하기 전 초기화 단계에서 '동작 전 자가시험'의 일환으로 무결성을 검증해야 합니다 1 2.

소스 문서에서는 이러한 무결성 검증 기법의 구체적인 구현 예시로 대칭키 기반의 **HMAC-SHA256**과 비대칭키(공개키) 기반의 **RSA-PSS**를 상세히 다루고 있으며, 이 두 기법은 암호키의 관리 방식과 검증 구조에서 뚜렷한 차이를 보입니다.

1. 대칭키 기반 무결성 기법: HMAC-SHA256

HMAC-SHA256을 이용한 무결성 검증은 암호모듈을 빌드할 때 산출된 바이너리 파일에 대해 32 바이트의 무결성 검증값(MAC)을 계산하여 모듈 끝에 연결하는 방식으로 이루어집니다 3 4.

- **키 관리의 특징 (분산 저장 및 마스킹):** 대칭키 기반인 HMAC은 검증값을 생성할 때와 확인할 때 **동일한 256비트 무결성 검증키(K)**를 사용합니다 3 4. 만약 이 키가 모듈 내부에 단순한 문자열 형태로 저장되어 노출된다면, 공격자가 모듈을 변조한 뒤 새로운 검증값을 계산하여 덧붙일 수 있는 치명적인 취약점이 발생합니다. 따라서 소스코드 내에 키를 하드코딩할 때는 단순 문자열 탐색이나 메모리 덤프를 통해 키가 노출되는 것을 막기 위해, **XOR 마스킹 및 인덱싱 분산 방식**을 사용하여 안전하게 은닉해 3 5.

스튜디오

AI 오디오 오버뷰

슬라이드 자료

동영상 개요

마인드맵

보고서

플래시카드

퀴즈

인포그래픽

데이터 표

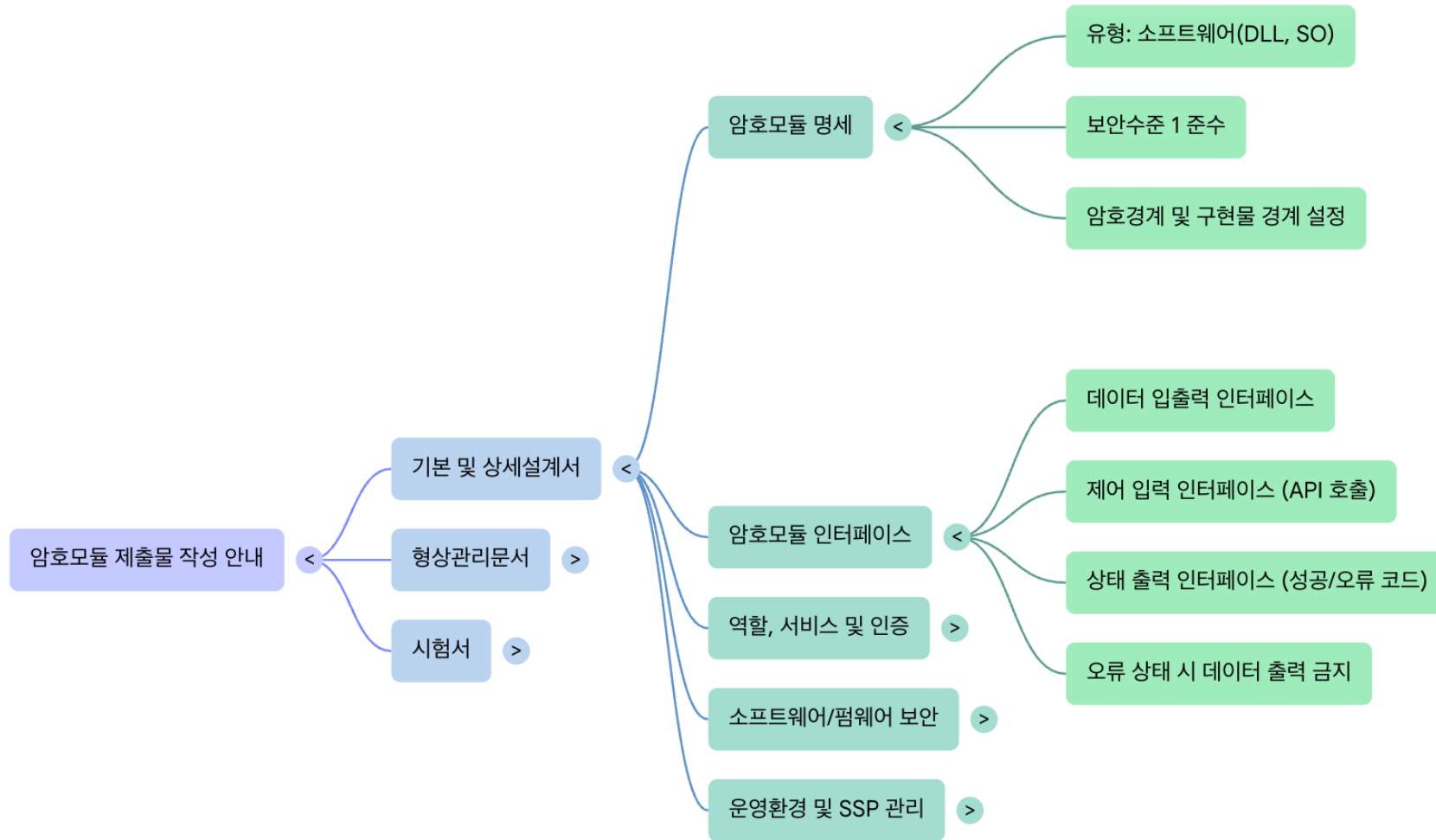
암호모듈 검증 제출물 작성 가이드라인

소스 3개 · 2분 전

부채널 공모전: AI 기반 KCMVP 사전 점검

33

Notebook LM 2



마크다운 뷰어

- IDE에서 돈보기 모양 클릭!!!!

LEA_코드_룰_작성_가이드.md

LEA 코드 룰 작성 가이드

LEA 코드 룰 작성 가이드 > ## 2) 룰 전체 구조와 작성 방법

Preview Markdown

1 ## LEA 코드 룰 작성 가이드

2

3 이 문서는 `backend/rules/algorithm/lea.yaml` 및 `backend/rules/mode/\*.yaml` (CBC/CCM/CFB/CMAC/CTR/ECB/GCM/OFB)에 들어가는

4 ****LEA 관련 코드 룰****을 어떻게 작성할지 정리한 가이드입니다.

5 구조는 `DOC\_설계서\_룰\_작성\_가이드.md`와 동일하게 맞추되, 대상이 ****코드(C 소스)****라는 점만 다릅니다.

6

7 ---

8

9 ### 1) LEA 코드 룰셋 개요

10

11 - ****목적****: LEA 알고리즘 및 그 운용모드(CBC, CTR, GCM 등)를 구현한 코드가

12 - 알고리즘/모드 스펙(KCMVP, 표준 문서)을 준수하는지,

13 - 위험한 사용 패턴(ECB 반복, Nonce/IV 재사용 등)을 사용하지 않는지

14 를 L1 룰(정규식 기반)으로 빠르게 검사하기 위함입니다.

15

16 - ****대상 파일****:

17 - LEA 블록 암호 구현부, 키 스케줄, 운용모드(CBC/CTR/...)를 구현한 C/헤더 파일들.

18 - 분석 시 API에서 `algorithm=LEA`, `mode=CBC`와 같이 지정한 값에 따라 해당 YAML이 로드됩니다.

19

20 - ****위치****:

21 - 알고리즘 공통: `backend/rules/algorithm/lea.yaml`

22 - 운용모드별: `backend/rules/mode/\*.yaml` (cbc/ccm/cfb/cmac/ctr/ecb/gcm/ofb)

23

24 ---

25

26 ### 2) 룰 전체 구조와 작성 방법

27

28 코드용 LEA 룰은 공통적으로 아래 필드들을 가집니다.

29

30 ### 룰 구조 (공통)

31

32 | 필드 | 필수 여부 | 설명 · 입력 방법 |

33 |-----|-----|

34 | ****id**** | 필수 | 룰 고유 ID. LEA 알고리즘은 `LEA-0xx`, 모드별은 `CBC-0xx`, `CTR-0xx` 등으로 구분 (예: LEA-001, GCM-002). |

35 | ****category**** | 필수 | `algorithm` 또는 `mode`. 알고리즘 공통 룰인지, 모드별 룰인지 구분. |

36 | ****algorithm / mode**** | 필수 | category에 따라 하나만 사용. `algorithm: LEA` 또는 `mode: CBC` 처럼 대상을 명시. |

37 | ****name**** | 필수 | 룰 이름(한글/영문). 위반 목록에 표시될 짧은 설명. |

LEA 코드 룰 작성 가이드

이 문서는 `backend/rules/algorithm/lea.yaml` 및 `backend/rules/mode/*.yaml` (CBC/CCM/CFB/CMAC/CTR/ECB/GCM/OFB)에 들어가는 LEA 관련 코드 룰을 어떻게 작성할지 정리한 가이드입니다. 구조는 `DOC_설계서_룰_작성_가이드.md`와 동일하게 맞추되, 대상이 **코드(C 소스)**라는 점만 다릅니다.

1) LEA 코드 룰셋 개요

- 목적: LEA 알고리즘 및 그 운용모드(CBC, CTR, GCM 등)를 구현한 코드가
 - 알고리즘/모드 스펙(KCMVP, 표준 문서)을 준수하는지,
 - 위험한 사용 패턴(ECB 반복, Nonce/IV 재사용 등)을 사용하지 않는지를 L1 룰(정규식 기반)으로 빠르게 검사하기 위함입니다.
- 대상 파일:
 - LEA 블록 암호 구현부, 키 스케줄, 운용모드(CBC/CTR/...)를 구현한 C/헤더 파일들.
 - 분석 시 API에서 `algorithm=LEA`, `mode=CBC`와 같이 지정한 값에 따라 해당 YAML이 로드됩니다.
- 위치:
 - 알고리즘 공통: `backend/rules/algorithm/lea.yaml`
 - 운용모드별: `backend/rules/mode/*.yaml` (cbc/ccm/cfb/cmac/ctr/ecb/gcm/ofb)

2) 룰 전체 구조와 작성 방법

코드용 LEA 룰은 공통적으로 아래 필드들을 가집니다.

룰 구조 (공통)

필드	필수 여부	설명 · 입력 방법
----	-------	------------

Cursor Tab 줄 27, 열 1 공백: 2 UTF-8 LF () Markdown